

Sorting Files

1. COBOL Sort
2. JCL Sort
3. Application of Sorted Files

Purpose of Sorting Data

- Improved data retrieval
- Data analysis
- Presentation of information
- Enhance decision-making

COBOL Sort

Syntax of SORT

SORT *work-file*

ON ASCENDING/DESCENDING KEY *key-field-1* [*key-field-2 ...*]

USING *input-file*

GIVING *output-file*

Components of the SORT statement

- ***input-file***: The file containing the unsorted records
- ***output-file***: The file where the sorted records are stored.
- ***key-field***: The field(s) used to determine the sorting order. The leftmost key-field is the primary key, the next key is the secondary key, and so on.
- ***work-file***: A temporary file used during the sort processing. It is defined in the SD section in COBOL program.

1. Describing the sort work file

1. Write SELECT clause under FILE-CONTROL to name a sort file. For example:

```
ENVIRONMENT DIVISION.  
INPUT-OUTPUT SECTION.  
FILE-CONTROL.  
SELECT Sort-Work ASSIGN TO SortWork.
```

Sort-Work is the name of the file in your program. Use this name to refer to the file.

2. Describe the sort file in an SD entry in the FILE SECTION. SD entry must contain a record description. For example:

```
DATA DIVISION.  
FILE SECTION.  
SD Sort-Work  
01 SORT-WORK-RECORD.  
    05 SORT-KEY-1 PIC X(10) .  
    05 SORT-KEY-2 PIC X(10) .  
    05 FILLER      PIC X(80) .
```

The file described in an SD entry is the working file used for a sort operation. You cannot perform any I/O on this file, i.e., OPEN, READ, WRITE, CLOSE.

2. Describing the input to sorting

1. Write SELECT clause under FILE-CONTROL to name the input file. For example:

```
ENVIRONMENT DIVISION.  
INPUT-OUTPUT SECTION.  
FILE-CONTROL.  
SELECT Input-File ASSIGN TO InFile.
```

Input-File is the name of the file in your program. Use this name to refer to the file.

2. Describe the input file (or files when merging) in an FD entry in the FILE SECTION. For example:

```
DATA DIVISION.  
FILE SECTION.  
FD Input-File  
01 Input-Record PIC X(100).
```

3. Describing the output from sorting

1. Write SELECT clause under FILE-CONTROL to name the output file. For example:

```
ENVIRONMENT DIVISION.  
INPUT-OUTPUT SECTION.  
FILE-CONTROL.  
SELECT Output-File ASSIGN TO OutFile.
```

Input-File is the name of the file in your program. Use this name to refer to the file.

2. Describe the output file (or files when merging) in an FD entry in the FILE SECTION. For example:

```
DATA DIVISION.  
FILE SECTION.  
FD Output-File  
01 Output-Record PIC X(100) .
```

4. Coding SORT statement

```
SORT Sort-Work  
  ON ASCENDING KEY Sort-Key-1 Sort-Key-2  
  USING Input-File  
  GIVING Output-File
```

Input-File and Output-file MUST not be open during the sort. The compiler generates statements to open, read/write and close the files.

5. Determining whether the sort or merge was successful

You should test for successful completion after each SORT statement. For example:

```
SORT Sort-Work  
  ON ASCENDING KEY Sort-Key-1 Sort-Key-2  
  USING Input-File  
  GIVING Output-File  
  
IF SORT-RETURN NOT = 0  
  DISPLAY 'SORT ENDED ABNORMALLY. SORT-RETURN = ' SORT-RETURN.
```

CLG JCL for COBOL Sort

```
//*
//*****
//*    COBOL SORT JCL
//*****
//*
//STEP1 EXEC IGYWCLG
//COBOL.SYSIN DD DSN=KC03???.COMP251.COBOL(SORTPGM),DISP=SHR
//*****          COBOL AND OS USES THESE FILES
//GO.CEEDUMP DD SYSOUT=*,OUTLIM=4000          PREVENT LARGE DUMPS
//*
//GO.SYSUDUMP DD SYSOUT=*,OUTLIM=2000          PREVENT LARGE DUMPS
//GO.SYSOUT DD SYSOUT=*,OUTLIM=2000          USED BY DISPLAY COMMAND
//*****          SORTWORK FILE
//GO.SORTWORK DD UNIT=SYSDA,SPACE=(TRK,(1,1)),DISP=(NEW,PASS)
//*****          SORTED STUDENT FILE, TEMP USE ONLY
//GO.OUTFILE DD UNIT=SYSDA,SPACE=(TRK,(1,1)),DISP=(NEW,PASS)
//*****          UNSORTED INSTREAM STUDENT DATA
//GO.INFILE DD *
JOE          STUDENT      5553
MARY         BROWN       5484
BILL         SMITH        7895
JACK         ARDEN        9876
BOB          SMITH        8767
JILL         MOORE        9828
MARGRET      BROWN       7659
DICK         DUNN         9210
FRANK        GRACE        7651
JO           STUDENT      8762
/*
//*
```


JCL Sort (DFSORT)

DFSORT is a high-performance utility developed by IBM for sorting, merging, copying, and analyzing data in mainframe environments. It is widely used in **Job Control Language (JCL)** to manage large datasets efficiently. DFSORT is also referred to as **SORT** and is an integral part of z/OS systems.

DD Names used in Sort JCL

- SORTIN
Unsorted input file
- SORTOUT
Sorted output file
- SYSIN
Contains Sort Control statement

JCL Sort (DFSORT)

Sort by last name and then first name

```
000001 //KC02134A JOB 1,
000002 // 'XXXXXXXXXX ', PUT YOUR NAME INSIDE THE QUOTES
000003 // MSGCLASS=H,
000004 // NOTIFY=&SYSUID,TIME=(,10)
000005 //*
000006 //*
000007 //* COMP251 - COBOL
000008 //*****
000009 //* JCL SORT - DFSORT *
000010 //*****
000011 //*
000012 //STEP010 EXEC PGM=IEFBR14
000016 //DD01 DD DSN=KC02134.COMP251.SORTOUT,
000017 // DISP=(MOD,DELETE,DELETE)
000018 //*
000019 //STEP020 EXEC PGM=SORT,REGION=0M
000020 //SYSOUT DD SYSOUT=* // OUTPUT MESSAGES FROM SORT
000021 //SORTIN DD DSN=KC02134.COMP251.SORTIN, // UN-SORTED INPUT
000022 // DISP=SHR
000023 //SORTOUT DD DSN=KC02134.COMP251.SORTOUT, // SORTED OUTPUT
000024 // DISP=(,CATLG,),
000025 // UNIT=SYSDA,
000026 // DCB=(RECFM=FB,LRECL=80,BLKSIZE=0),
000027 // SPACE=(400,8)
000028 //SYSIN DD * // SORT CONTROL STATEMENT
000029 *** COL 1 - 10 FIRST NAME
000030 *** COL 11 - 20 LAST NAME
000031 *** COL 21 - 4 STUDENT ID
000032
000033 SORT FIELDS=(11,10,CH,A,1,10,CH,A) // SORT BY LAST NAME AND THEN FIRST NAME
000034 // POSITION,LENGTH,TYPE,ORDER
000035 /*
```

← Must leave a space

JCL Sort (DFSORT)

Sort Control Statement

SORT FIELDS=(*starting position, length (in bytes), data type, order*)

starting position – starting position of sort field in the record

length (in bytes) – length of sort field in bytes

data type – data type of sort field

CH – character

ZD – Zoned Decimal

PD – Packed Decimal

BI – Binary

order – sort sequence

A – ascending

D – descending

JCL Sort (DFSORT)

Sort by last name and then first name

```
VIEW          KC02134.COMP251.SORTIN          Columns 00001 00072
Command ==>          Scroll ==> CSR
=COLS>  ----+----1----+----2----+----3----+----4----+----5----+----6----+----7--
*****  ***** Top of Data *****
000001 JOE          STUDENT      5553
000002 MARY          BROWN       5484
000003 BILL          SMITH        7895
000004 JACK          ARDEN        9876
000005 BOB           SMITH        8767
000006 JILL          MOORE        9828
000007 MARGRET       BROWN       7659
000008 DICK          DUNN        9210
000009 FRANK         GRACE        7651
000010 JO           STUDENT      8762
*****  ***** Bottom of Data *****

VIEW          KC02134.COMP251.SORTOUT         Columns 00001 00072
Command ==>          Scroll ==> CSR
=COLS>  ----+----1----+----2----+----3----+----4----+----5----+----6----+----7--
*****  ***** Top of Data *****
000001 JACK          ARDEN        9876
000002 MARGRET       BROWN       7659
000003 MARY          BROWN       5484
000004 DICK          DUNN        9210
000005 FRANK         GRACE        7651
000006 JILL          MOORE        9828
000007 BILL          SMITH        7895
000008 BOB           SMITH        8767
000009 JO           STUDENT      8762
000010 JOE           STUDENT      5553
*****  ***** Bottom of Data *****
```

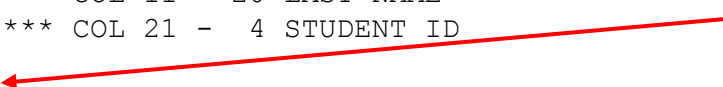
Column Description
1-10 First name
11-20 Last name
21-24 Student ID

`SORT FIELDS=(11,10,CH,A,1,10,CH,A)`

JCL Sort (DFSORT)

Sort by Student ID (Zoned Decimal)

```
000001 //KC02134A JOB 1,
000002 // 'XXXXXXXXXX ', PUT YOUR NAME INSIDE THE QUOTES
000003 // MSGCLASS=H,
000004 // NOTIFY=&SYSUID,TIME=(,10)
000005 //*
000006 //*
000007 //* COMP251 - COBOL
000008 //*****
000009 //* JCL SORT - DFSORT *
000010 //*****
000011 //*
000012 //STEP010 EXEC PGM=IEFBR14
000016 //DD01 DD DSN=KC02134.COMP251.SORTOUT,
000017 // DISP=(MOD,DELETE,DELETE)
000018 //*
000019 //STEP020 EXEC PGM=SORT,REGION=0M
000020 //SYSOUT DD SYSOUT=* // OUTPUT MESSAGES FROM SORT
000021 //SORTIN DD DSN=KC02134.COMP251.SORTIN, // INPUT DATASET
000022 // DISP=SHR
000023 //SORTOUT DD DSN=KC02134.COMP251.SORTOUT,
000024 // DISP=(,CATLG,),
000025 // UNIT=SYSDA,
000026 // DCB=(RECFM=FB,LRECL=80,BLKSIZE=0),
000027 // SPACE=(400,8)
000028 //SYSIN DD * // SORT CONTROL STATEMENT
000029 *** COL 1 - 10 FIRST NAME
000030 *** COL 11 - 20 LAST NAME
000031 *** COL 21 - 4 STUDENT ID
000032
000033 SORT FIELDS=(21,4,ZD,A) // SORT BY STUDENT ID (ZONED DECIMAL)
000034 // POSITION,LENGTH,TYPE,ORDER
000035 /*
```



Must leave a space

JCL Sort (DFSORT)

Sort by Student ID (Zoned Decimal)

```
VIEW          KC02134.COMP251.SORTOUT          Columns 00001 00072
Command ==>          Scroll ==> CSR
=COLS>  ----+-----1-----+-----2-----+-----3-----+-----4-----+-----5-----+-----6-----+-----7--
*****  ***** Top of Data *****
000001 MARY      BROWN      5484
000002 JOE       STUDENT    5553
000003 FRANK     GRACE      7651
000004 MARGRET   BROWN      7659          SORT  FIELDS=(21,4,ZD,A)
000005 BILL      SMITH      7895
000006 JO        STUDENT    8762
000007 BOB       SMITH      8767
000008 DICK      DUNN       9210
000009 JILL      MOORE      9828
000010 JACK      ARDEN      9876
*****  ***** Bottom of Data *****
```

Application of Sorted Files (File Matching)

One of the applications of sorted files is file matching. This technique is highly used in the industry for file processing.

Master File

- contains the core data elements that define and categorize business entities, such as customers, products, and suppliers.

Transaction File

- captures the detailed records of individual business transactions, such as sales orders, purchase orders, and invoices.

Case Study

Update customer bank account balance based on daily banking transactions.

Customer Bank Account File (Master File)	Daily Transaction File
1. Account number (unique number) 2. Account Balance	1. Account number (unique number, no duplicates in the file) 2. Transaction Type (W-withdrawal, D-Deposit) 3. Transaction Amount

Solution Approach

In order to update Customer File efficiently, both files are first sorted in the same sequence by Account Number. Then a file matching technique will be deployed to update Account Balance based on daily transaction amount.

Application of Sorted Files (File Matching)

Solution Design

Steps:

- Sort both files by Account Number in ascending sequence. A JCL sort will be an efficient tool to accomplish that.
- Write a program to match both files by Account Number. When there is a match on Account Number, update Account Balance based on transaction amount .

Pseudo-Code for File Matching

Define input files: (1) IN-CUST (old master file), (2) IN-TRAN

Define output file: (1) OUT-CUST (new master file)

Logic:

```
READ IN-CUST
READ IN-TRAN
PERFORM UNTIL EOF-IN-CUST OR EOF-IN-TRAN
  IF ACCT-NUM OF IN-CUST = ACCT-NUM OF IN-TRAN
    COMPUTE ACCT-BALANCE = ACCT-BALANCE +/- TRAN-AMOUNT
    WRITE OUT-CUST
    READ IN-CUST
    READ IN-TRAN
  ELSE
    IF ACCT-NUM OF IN-CUST < ACCT-NUM OF IN-TRAN
      WRITE OUT-CUST FROM IN-CUST
      READ IN-CUST
    ELSE
      READ IN-TRAN
    END-IF
  END-IF
END-IF
END-PERFORM

IF EOF-IN-TRAN      /* WRITE REMAINING OLD MASTER FILE RECORDS TO NEW MASTER FILE
  PERFORM UNTIL EOF-IN-CUST
    WRITE OUT-CUST FROM IN-CUST
    READ IN-CUST
  END-PERFORM
END-IF
```

Scenario	
ACCT-NUM OF IN-CUST	ACCT-NUM OF IN-TRAN
1	1
2	3
3	4
5	
6	

Application of Sorted Files (File Matching)

Alternate solution – using a table

1. Load the Customer Account File (old master file) in a table indexed by Account Number.
2. For each transaction in the daily file, do a binary search on the table by Account Number and update Account Balance when there is a match.
3. After all transactions in the daily file are matched, write all table entries in the new Master File.

This solution might be less complexed in terms of logic. However, if the Customer Account File contains large number of records (a typical Customer Account File is in millions of records), memory usage will be a concern. Also, table size, i.e. statement in **OCCURS ? TIMES**, will need to be re-adjusted regularly.